

Informe Completo de Optimización: Pipeline SD

+ AMD Vega 64

Guía de Parámetros, Ajustes de Inferencia y Arquitectura del Juego

1. Configuración de Hardware e Infraestructura Local

Para lograr un entorno de inferencia continuo en tiempo real sin cuellos de botella de memoria o alimentación:

- División de Cargas (iGPU + dGPU):** La gráfica integrada Ryzen 2400G asume la interfaz de escritorio de Kubuntu. La AMD RX Vega 64 (8GB HBM2) queda totalmente libre para alojar el servidor de difusión y el LLM local sin competir por VRAM.
- Descarte de Alimentación Secundaria:** Se confirmó que la Vega 64 maneja ambas cargas de IA sin superar el umbral térmico ni eléctrico de la fuente principal, evitando la necesidad de usar adaptadores como *ADD2PSU* o fuentes auxiliares.
- Backend Servidor:** Utilización del binario en C/C++ sd-server compilado con soporte Vulkan para exprimir la arquitectura GCN5 de la Vega 64 en Linux.

2. Selección de Modelo, LCM y Cuantización

- Checkpoint Base:** ComicCraftLCM_beta4.safetensors en formato FP16, manteniendo una huella de memoria extremadamente compacta (~2.03 GB VRAM).
- Aceleración LCM (Latent Consistency Model):** Al integrar pesos LCM, se redujo drásticamente la cantidad de pasos de muestreo requeridos para formar una imagen coherente (de los 20-30 pasos habituales a tan solo 4 pasos).

3. Parámetros de Inferencia y Muestreador (Sampler Tuning)

Se realizaron ajustes en las variables de muestreo para conseguir tiempos sub-2.5s por imagen:

Parámetro	Valor Ajustado	Efecto en Rendimiento y Calidad
Sampler / Scheduler	Euler A + CFG++ / Karras	Muestreo ultra-rápido que converge en pocas pasadas. Evita distorsiones en composiciones de

Parámetro	Valor Ajustado	Efecto en Rendimiento y Calidad
		pocos pasos.
Pasos de Muestreo (Steps)	4 pasos	Reduce el tiempo de cálculo del UNet al mínimo indispensable gracias al entrenamiento LCM.
CFG Scale (Guidance)	1.5 - 2.0	Valores reducidos específicos para LCM, evitando artefactos o saturación excesiva de color.
Alineación Múltiples de 64	Ej. 320x448, 384x512	Evita advertencias de reescalado dinámico (<i>align generation request</i>) en <i>stable-diffusion.cpp</i> , aprovechando los tensores nativos de la GPU.

4. Evolución de Benchmark por Resolución

Resolución	Píxeles Totales	Sampling (1/1)	VAE Decode	Tiempo Total	Velocidad
512 x 512	262.144	~2.60s	~0.80s	~3.40s	1.90 it/s
384 x 384	147.456	1.69s	0.41s	2.15s	2.38 it/s
320 x 448	143.360	1.66s	0.38s	2.10s	2.41 it/s
384 x 512	196.608	2.06s	0.53s	2.65s	1.95 it/s

5. Técnica de Inyección de Prompts (Layered Prompting)

Dado que los checkpoints de ilustración tienden a estar sesgados hacia rostros pulidos y figuras femeninas, se estableció la estrategia de inyección procedimental desde el LLM:

- **Base Fija / Estilo:** Coletilla de estilo gráfico general (ej. gritty comic style, high contrast, ink shading).
- **Capa de Contexto / Criaturas:** Para superar la incapacidad del modelo con zombis/monstruos, el LLM inyecta términos de textura dura: ghoul, decayed flesh, pale skin, dark shadows, horror.
- **Filtro Negativo:** Inclusión de card frame, border, text, watermark para forzar ilustraciones limpias "a

sangre".

6. Arquitectura Móvil (Android UI Layout)

Para no gastar recursos computacionales de la IA en marcos simétricos ni texto ilegible:

- **Capa 1 (Fondo):** Imagen cruda de SD renderizada a 320×448 o 384×512 .
- **Capa 2 (Intermedia):** Marco estático en PNG transparente con textura de carta/pergamino superpuesto en Android.
- **Capa 3 (Texto Vectorial):** Título y estadísticas renderizados con la tipografía nativa de Android sobre el marco.